
Path Repeater

Full Project Breakdown — From Idea to Output

2025 Azerbaijan Grand Prix | DUS3R / MonST3R | Python, PyTorch, Open3D

Contents

1. **Project Goal and Problem Statement**
 2. Why This Problem Is Hard
 3. Literature Review and Model Selection
 4. The Raw Dataset — What We Started With
 5. Preprocessing — Step by Step
 6. Chunking Strategy and Memory Management
 7. The DUS3R Network — Architecture and Inference
 8. Global Alignment — Making a Chunk Consistent
 9. The Confidence Mask — Binary Encoding
 10. Chunk Stitching — Building the Global Map
 11. MonST3R — The Dynamic Scene Extension
 12. What Went Wrong — Bugs and Fixes
 13. Results — What We Produced
 14. Lessons Learned
 15. Future Work
-

1. Project Goal and Problem Statement

The aim of the VROOM project is to reconstruct the Monaco Formula 1 circuit as a navigable 3D point cloud, using nothing but the onboard camera footage from a single racecar. The reconstruction must also recover the camera trajectory — the path the car took through the lap — which corresponds to the optimal racing line.

Formally, given a monocular video $V = \{I_0, I_1, \dots, I_N\}$ recorded from a camera rigidly attached to a moving vehicle, the system must produce:

- **A 3D point cloud P** — a dense set of 3D points representing the static geometry of the circuit: track surface, barriers, grandstands, and buildings.
- **A camera trajectory T** — the sequence of camera poses $\{T_0, T_1, \dots, T_N\}$ where each T_i in $SE(3)$ gives the position and orientation of the camera at frame i .

No external information is available: no GPS, no IMU, no LiDAR, no stereo camera, no known camera intrinsics, no pre-built map. The system must infer everything from pixel values alone.

This is an instance of Simultaneous Localisation and Mapping (SLAM) combined with dense 3D reconstruction. Classical SLAM solves for sparse pose graphs; we require a dense reconstruction of the full scene, which is a significantly harder problem.

2. Why This Problem Is Hard

Formula 1 onboard footage violates almost every assumption made by standard SLAM and 3D reconstruction systems. This section documents each challenge and explains its technical consequence.

2.1 High-Speed Motion

Formula 1 cars corner at speeds up to 300 km/h and accelerate from 0 to 100 km/h in under 2 seconds. The inter-frame baseline at 50 fps and 200 km/h is approximately 1.1 metres. This is not excessive, but at tighter corners where the car is also rotating rapidly, the angular motion is extreme. Classical feature matching (SIFT, ORB) relies on the assumption that the image region around a keypoint looks similar in two consecutive frames. When the camera rotates by 30 degrees between frames, this assumption fails and matches cannot be found.

2.2 The Static Car Body Problem

The car's chassis, halo bar, steering wheel, mirror housings, and driver helmet occupy the lower 40 to 50 percent of every single frame. They never move relative to the camera because they are mechanically attached to it. Any reconstruction method that does not explicitly remove or ignore these pixels will reconstruct them as a large, static 3D object — one that appears to follow the camera trajectory everywhere. The result is the characteristic 'red cylinder' artefact (red from the Ferrari livery) that runs along the full trajectory, obscuring the track geometry.

2.3 Monocular Scale Ambiguity

A single camera has no way to determine the absolute scale of the scene. If you scale all 3D points by a factor of 2 and halve all camera translations, the images look identical. This means every monocular reconstruction is correct only up to an unknown scale factor. Combining multiple independently-reconstructed segments — which each have their own unknown scale — requires explicit scale normalisation, which is mathematically non-trivial.

2.4 Dynamic Scene Elements

In the Monaco Grand Prix, 20 cars race simultaneously. On lap 1 the field is tightly packed, meaning other cars are visible ahead of the camera for significant portions of the lap. A reconstruction system that assumes a static world will try to place these cars in the point cloud, corrupting the track geometry at exactly the points where good reconstruction matters most — the corners where cars are closest together.

2.5 GPU Memory Limits

A full lap at 50 fps produces over 5,500 frames. Even at 24 fps after subsampling, 2,640 frames remain. DUS_t3R processes all pairs of frames in a chunk simultaneously, with memory scaling as $O(n^2)$ in the number of frames. No available GPU can hold all 2,640 frames and their pairwise computations at once. The video must be split into small segments, which introduces the stitching problem.

2.6 Unknown Camera Intrinsics

The camera's intrinsic parameters — focal length, principal point, and lens distortion — are not provided with the broadcast footage. DUS_t3R must estimate these jointly with the 3D structure during global alignment. This is more difficult than the case where intrinsics are known and only extrinsics need to be estimated.

3. Literature Review and Model Selection

3.1 Classical SLAM Approaches Considered

We evaluated several classical SLAM systems before choosing DUS3R.

- **ORB-SLAM3** — Orb-SLAM3 uses ORB keypoints and descriptors with bundle adjustment. It is highly accurate on standard benchmarks (TUM RGB-D, EuRoC) but requires well-textured scenes and moderate camera motion. On F1 footage at high speed, the keypoint matching stage fails when features move too far between frames. We found that ORB-SLAM3 lost tracking within seconds of attempting to process the onboard video.
- **LSD-SLAM** — LSD-SLAM is a direct method that uses image gradients rather than keypoints. It performs better on low-texture scenes but is sensitive to fast camera rotation and assumes near-constant illumination. The dramatic lighting changes when the Monaco circuit passes through tunnels caused repeated tracker resets.
- **COLMAP** — COLMAP is a Structure-from-Motion system, not a real-time SLAM system. It is highly accurate for static scenes with sufficient image overlap but requires all frames to be processed offline and does not handle dynamic objects. On our dataset it produced good results for individual short segments but could not be scaled to a full lap without prohibitive compute time.

3.2 Learning-Based Methods

- **SuperPoint + SuperGlue** — These learned feature detection and matching networks are more robust than classical methods at high speed. They improve on ORB matching substantially but still rely on explicit correspondences. When the camera motion is large enough that the same physical point is not visible in consecutive frames, even SuperGlue fails.
- **DROID-SLAM** — DROID-SLAM introduced differentiable bundle adjustment with learned optical flow. It was our first choice and is referenced in the paper. However, it required a display/GUI to visualise outputs during development, which was not available on our compute environment, and even without dynamic objects it tends to reconstruct only the static elements of a scene. We abandoned it when DUS3R became available.
- **DUS3R** — DUS3R (Dense Unconstrained Stereo 3D Reconstruction) removes the need for explicit feature matching entirely. A vision transformer processes two frames jointly via cross-attention and directly predicts the 3D coordinates of every pixel in both frames. This makes it robust to the large inter-frame motions of F1 footage. We selected DUS3R as our primary reconstruction engine.
- **MonST3R** — MonST3R (Motion and Structure in 3D) extends DUS3R specifically for dynamic scenes by incorporating optical flow and predicting per-pixel motion masks. It directly addresses the moving-car problem. The paper's results were obtained primarily with MonST3R.

3.3 Selection Rationale

We chose DUS3R as the starting point because it: (a) requires no explicit feature matching, making it robust to fast motion; (b) predicts dense pointmaps for all pixels, not just detected keypoints, giving much richer 3D reconstructions; (c) handles unknown camera intrinsics natively; and (d) is available as a pre-trained model requiring no fine-tuning on F1 footage.

MonST3R was identified as the ideal upgrade for handling other cars in the scene, matching the approach described in the original VROOM paper.

4. The Raw Dataset — What We Started With

4.1 Source and Content

The dataset consists of onboard broadcast video from the 2023 Monaco Grand Prix, sourced from F1 TV. We obtained 20 videos — one per driver — for the full duration of the race. For our primary experiments we focused on a single driver's video, specifically the first 110 seconds of the race covering approximately one full lap of the 3.337 km Monaco circuit.

The Monaco circuit was chosen specifically because its combination of tight corners, close walls, and rich trackside texture (casino, harbour, tunnel) provides both challenging and informative reconstruction scenarios. The tight walls in particular yield high-quality optical flow because the scene surface is close to the camera.

4.2 Technical Specifications of the Raw Video

Property	Value
Codec	H.264 (AVC), High profile
Resolution	1920 x 1080 pixels (Full HD)
Frame rate	50 fps
Duration	110 seconds (first lap)
File size (raw)	~450 MB per driver
Colour space	YUV 4:2:0, BT.709
Bitrate	~9,000 kbps
Audio	AAC (irrelevant — discarded)
Camera intrinsics	Unknown (no calibration data)
GPS / IMU	Not available (broadcast feed only)

4.3 What the Video Shows

The video is a forward-facing view from approximately 60 cm above the track surface. The field of view is wide (approximately 120 degrees horizontally), capturing the track ahead, barriers on both sides, and the sky above. Other cars are visible ahead of the camera for approximately 35 to 40 percent of the footage during the first lap when the field is still tightly packed.

The bottom 45 to 50 percent of every frame is dominated by the red Ferrari chassis, the halo bar across the top of the frame interior, the steering wheel in the centre, and the driver's gloved hands. The driver's helmet is partially visible at the bottom centre. None of these elements move relative to the camera throughout the entire video.

5. Preprocessing — Step by Step

Preprocessing transforms the raw video into a form the reconstruction network can use effectively. Each step addresses a specific failure mode. The order of operations matters and was determined empirically.

5.1 Trimming

We trimmed the video to the first 1 minute 50 seconds. The original footage of 2 minutes 14 seconds includes a period after the car reaches the end of the pit straight where the camera cuts to a different angle. This camera cut is fatal for SLAM methods because the abrupt scene change cannot be interpreted as camera motion. We remove it by hard-cutting the video at 1:50.

Tool: ffmpeg with the -t flag using stream copy (-c copy) to avoid re-encoding, preserving the original quality exactly.

5.2 Downscaling

The raw 1920x1080 frames are too large for GPU processing. The DUS3R network internally resizes its input to 512 pixels on the longest side regardless of the provided resolution, so there is no benefit to supplying 1920-wide frames. We resize to 1024x576 as a preprocessing step, reducing the file sizes and load times without losing any information the network would have used.

We applied bilinear interpolation using ffmpeg's scale filter: scale=1024:576. The choice of 1024 (rather than 512 directly) preserves more quality in the JPEG compression step that follows, since the network will resize to 512 internally — applying two successive downscales is slightly better than one large downscale.

5.3 Frame Rate Reduction

The 50 fps video is reduced to approximately 24 fps by extracting every second frame (stride k=2). This was determined experimentally by testing several frame rates and measuring reconstruction quality:

At 50 fps the frames are too similar — the network finds many redundant pairs and the reconstruction adds little new information for each additional pair processed. At rates below 12 fps the inter-frame motion exceeds the optical flow estimator's tracking range and reconstruction breaks down entirely. 24 fps gives a good balance of inter-frame diversity and trackable motion.

5.4 Car Body Removal — The Most Critical Step

This is the single most impactful preprocessing decision in the project. Removing the car body correctly is what separates a usable reconstruction from an artefact-ridden failure.

First Attempt: Bottom-Half Masking

Our first approach was to set all pixels in the lower half of the frame to black (zero intensity). The intuition was correct — remove the car — but the implementation was wrong. The reconstruction network receives a 512-wide frame where 50% of pixels are exactly zero. The network's confidence predictor correctly identifies these constant-zero regions as having no useful 3D structure and assigns them the minimum confidence value. Those pixels contribute nothing to the reconstruction but still occupy half the network's input budget. On sections of the track with low texture in the upper half (long straights, the tunnel), the remaining valid pixels also had insufficient texture, and entire chunks produced zero output points.

WARNING: Masking produced zero-point chunks on approximately 35% of the video. On these sections of the lap, no 3D geometry was recovered at all.

Second Attempt: Top-Half Cropping

The correct approach is to physically remove the lower half of each frame rather than zeroing it. The cropped frame is 1024x288 — a wide, short image containing only the track ahead, barriers, and environment. Every pixel

the network receives carries real scene information. The network's capacity is used entirely for valid reconstruction.

RESULT: After switching to cropping: chunks 0, 1, and 2 immediately produced 154,000, 132,000, and 104,000 points respectively. The 35% zero-point-chunk rate dropped to near zero for all but the most featureless track sections.

Why This Matters Mathematically

The global alignment energy E weights each pixel by its confidence $C(u,v)$. Masked pixels have $C(u,v)$ approx 1 (minimum). Cropped pixels don't exist in the input, so they don't consume any weight in E . The effective signal-to-noise ratio of the alignment energy doubles when using cropping instead of masking.

5.5 Frame Extraction

After video preprocessing (trim, scale, crop), we extract individual frames as JPEG images at 24 fps. We used quality level 95 for JPEG compression to minimise compression artefacts while keeping file sizes manageable. Each frame is saved with a zero-padded five-digit index: frame_00000.jpg, frame_00001.jpg, and so on. The total extracted frame count for the 110-second video at 24 fps is 880 frames.

6. Chunking Strategy and Memory Management

6.1 Why Chunking Is Necessary

DUSt3R processes frame pairs. For a set of n frames it generates $n*(n-1)$ pairs (all pairs in both directions). Each pair requires a full forward pass through the 24-layer vision transformer encoder and 12-layer decoder. On a 16 GB T4 GPU, the maximum chunk size before running out of memory is approximately $n=12$ to 15 frames. Our 880-frame video must therefore be divided into many small windows.

6.2 The Chunking Parameters

We chose chunk size $c=20$ and overlap $o=5$, giving a stride of $s=c-o=15$. With 880 frames total and these parameters:

$$\begin{aligned} \text{Number of chunks} &= \text{ceil}((N - c) / s) + 1 = \text{ceil}((880 - 20) / 15) + 1 \\ &= 58 \text{ chunks} \end{aligned}$$

In our experiments with every-third-frame subsampling on one video we obtained 88 chunks. The overlap of 5 frames means every pair of consecutive chunks shares 5 frames, which are processed independently in each chunk and then used as registration anchors during stitching.

6.3 Naive vs Smart Chunking

Naive chunking splits at fixed frame intervals. The problem is that a corner may be split across two chunks, with the entry frames in one chunk and the exit frames in the next. The network then sees only half the corner in each chunk and cannot accurately reconstruct its curvature.

Smart chunking identifies straight sections of track (low angular velocity frames) and places chunk boundaries only there. A corner is always contained entirely within a single chunk with enough straight-track context on each side. We implemented this by computing the approximate angular velocity from consecutive frame optical flow magnitudes and splitting only when the angular velocity fell below a threshold.

6.4 The Overlap as Stitching Anchor

The $o=5$ overlapping frames between consecutive chunks are the mechanism that allows stitching. These frames appear in both chunk j and chunk $j+1$, are reconstructed independently in each chunk's local coordinate frame, and then the correspondence between their two sets of local poses is used to compute the transform that aligns chunk $j+1$ to chunk j 's already-established world frame. This is described in detail in Section 10.

7. The DUS3R Network — Architecture and Inference

7.1 Architecture Overview

DUS3R is a Siamese Vision Transformer (ViT) that takes two frames as input and produces two full-resolution 3D pointmaps as output. The architecture has three components: a shared encoder, a cross-attention decoder, and two output heads.

7.2 The Encoder

Both input frames I_a and I_b are processed by the same encoder (shared weights). The encoder is a ViT-Large:

- **Patch size** — 16x16 pixels. Each frame is divided into $(H/16) \times (W/16)$ patches.
- **Embedding dimension** — 1024. Each patch is linearly projected to a 1024-dimensional token.
- **Depth** — 24 transformer layers, each with 16-head self-attention.
- **Positional encoding** — RoPE100 (Rotary Position Embedding with 100 frequency bands). This encodes the 2D spatial position of each patch.

The encoder output is two sequences of patch tokens: one for I_a and one for I_b . Each sequence has length $(H/16) \times (W/16)$ with embedding dimension 1024.

7.3 The Cross-Attention Decoder

The decoder has 12 transformer layers with two key modifications over standard ViT decoders. First, it uses cross-attention: tokens from frame a attend to tokens from frame b and vice versa. This allows the decoder to reason about the spatial relationship between the two views — it can identify which patch in frame b corresponds to a given patch in frame a. This cross-frame communication is what enables 3D reconstruction without explicit feature matching.

Second, the decoder processes both sequences simultaneously, maintaining a single coherent representation of the shared scene geometry rather than treating the two frames independently.

7.4 The Output Heads

Two DPT (Dense Prediction Transformer) heads produce per-pixel predictions at the full input resolution (upsampled from the patch grid). Each head produces a 3-channel pointmap and a 1-channel confidence map:

- **Head a** — produces X_a in $\mathbb{R}^{H \times W \times 3}$ and C_a in $\mathbb{R}^{H \times W}$, both in I_a 's camera frame.
- **Head b** — produces X_b in $\mathbb{R}^{H \times W \times 3}$ and C_b in $\mathbb{R}^{H \times W}$, both also in I_a 's camera frame.

The fact that both pointmaps are in I_a 's frame is enforced during training by supervising both outputs against ground truth 3D point positions expressed in the I_a camera coordinate system.

7.5 Model Scale

The ViT-Large model we used has approximately 300 million parameters. The checkpoint file is 2.3 GB. At inference, the model requires approximately 4-6 GB of VRAM for a single pair of 512-pixel frames. Processing 132 pairs in global alignment for a 12-frame chunk requires 6-10 GB, fitting on the T4 (16 GB) but exceeding the GTX 1650 (4 GB) without reducing image size.

7.6 Inference Procedure

For each chunk of n frames we:

1. Load all n frames and generate all $n \times (n-1)$ symmetric pairs.
2. Run each pair through the DUS3R encoder and decoder to get (X_a, X_b, C_a, C_b) .

-
3. Store all pointmaps and confidence maps in memory for the global alignment step.
 4. Run global alignment (Section 8) to produce the final chunk reconstruction.

8. Global Alignment — Making a Chunk Consistent

8.1 The Problem

After running inference on all pairs in a chunk, we have a collection of pointmaps each in a different local coordinate frame. The pointmap $X^{\{ab\}}$ from pair (a,b) gives the scene in I_a 's frame; the pointmap $X^{\{cd\}}$ from pair (c,d) gives the scene in I_c 's frame. These are different frames. Simply concatenating all pointmaps into one cloud places each one at a different, arbitrary origin — the result is an incoherent blob.

WARNING: Skipping global alignment and using raw pointmaps directly was the primary bug in our original implementation. The output was a diagonal smear — visually indistinguishable from random noise.

8.2 The PointCloudOptimiser

DUST3R's PointCloudOptimiser (PCO) solves the global alignment problem by finding camera poses T_i in $SE(3)$ for each frame, per-pair scales $\sigma_{\{ij\}}$, and canonical 3D points χ that minimise a confidence-weighted reprojection energy. The PCO is implemented as a PyTorch module whose parameters are the poses and scales.

8.3 Initialisation Strategy

Starting Adam from a random initialisation would likely converge to a poor local minimum. The Minimum Spanning Tree (MST) initialiser builds an initial solution by:

1. Constructing a graph where nodes are frames and edge weights are the mean confidence of each pair.
2. Finding the maximum spanning tree (highest-confidence pairs first).
3. Traversing the tree from the root frame, propagating poses frame by frame using the pointmap geometry.

This gives a reasonable initial pose estimate that is close enough to the true solution for Adam to converge reliably.

8.4 Optimisation Details

Adam runs for 300 iterations with initial learning rate 0.01 and cosine decay. We monitored the loss value at each iteration. Typical convergence behaviour:

- **Iterations 0-50** — Rapid initial decrease. Loss drops by 60-70% of its initial value.
- **Iterations 50-150** — Slower refinement. Rotation and translation estimates stabilise.
- **Iterations 150-300** — Fine-tuning. Loss decreases slowly. Scale estimates converge.

Final loss values below 0.010 indicate good alignment. Values above 0.020 suggest the chunk contains challenging content (fast motion, low texture) and the reconstruction may be unreliable.

8.5 Output of Global Alignment

After optimisation, we extract from the PCO:

- **Camera poses** — $\{T_i\}$ in $SE(3)$ — one 4x4 matrix per frame, placing each frame in the chunk's consistent coordinate frame.
- **Canonical pointmap** — χ — the fused 3D point cloud for the chunk.
- **Confidence values** — $\{C^{\{ij\}}\}$ — used to compute the binary confidence mask M .
- **Camera intrinsics** — The focal length estimated per-frame during alignment.

9. The Confidence Mask — Binary Encoding

9.1 From Continuous to Binary

The global alignment produces continuous confidence values $C(u,v)$ for every pixel in every frame. To build the final point cloud, we must decide which pixels to include. We apply a percentile threshold to convert the continuous confidence map into a binary mask:

$$\begin{aligned} M(u,v) &= 1 && \text{if } C(u,v) \geq \text{Quantile_tau}(C_chunk) \\ M(u,v) &= 0 && \text{otherwise} \end{aligned}$$

This binary mask M is the 0-and-1 encoding of the data. Pixels where $M=1$ contribute a 3D point to the output cloud. Pixels where $M=0$ are discarded.

9.2 Choosing the Threshold Tau

We experimented with several values of tau:

- **tau = 0.5** — Keep only the top 50% most confident pixels. Produces clean, sparse point clouds on textured sections but zero points on low-texture sections. Not suitable for our dataset.
- **tau = 0.3** — Keep the top 70% of pixels. Better coverage but still misses low-texture straight sections.
- **tau = 0.0** — Keep all pixels regardless of confidence. Maximises coverage. Some sky and uniform-road points are included, but these are removed by the subsequent voxel downsampling step.

We settled on tau=0.0 (keep everything) followed by voxel downsampling at 0.01 units. This proved more robust than attempting to choose a specific confidence threshold.

9.3 Post-Filter: Voxel Downsampling

After binary selection with M , the point cloud may contain multiple points from different frames that represent the same physical surface point. Voxel downsampling divides 3D space into a regular grid of cubes (voxels) of side length $d=0.01$ reconstruction units, and replaces all points within each voxel with their centroid. This removes spatial duplicates, reduces the total point count by 60-80%, and produces a more uniform point density across the cloud.

10. Chunk Stitching — Building the Global Map

10.1 The Stitching Problem

Each chunk is reconstructed in an arbitrary local coordinate frame with an arbitrary scale. Chunk 0 might have its camera starting at position (0,0,0) facing the z-axis; chunk 1 might have its camera starting at (0,0,0) facing a completely different direction. The two local frames are unrelated. We need to find the transform that places each chunk in a common global frame.

10.2 The Overlap Mechanism

The $o=5$ overlapping frames are the key. Frame $f_{\{j*s+c-o\}}$ through $f_{\{j*s+c-1\}}$ appear in both chunk j and chunk $j+1$. In chunk j 's reconstruction, these frames have poses $T^{\{j\}}_{\{...\}}$ in chunk j 's coordinate frame. In chunk $j+1$'s reconstruction, the same physical frames have poses $T^{\{j+1\}}_{\{...\}}$ in chunk $j+1$'s coordinate frame. The pose of the first overlap frame of chunk $j+1$ in the global frame should equal the pose of the last overlap frame of chunk j in the global frame (since they represent the same camera position).

10.3 Rigid Weld

Given the last overlap frame's world pose $T^{\{world\}}_{\{i^*\}}$ (from chunk j) and the first overlap frame's local pose $T^{\{local\}}_{\{i^*\}}$ (in chunk $j+1$'s frame), the alignment transform is:

$$T_{align} = T^{\{world\}}_{\{i^*\}} * (T^{\{local\}}_{\{i^*\}})^{-1}$$

This is then applied to all poses and all points of chunk $j+1$:

$$\begin{aligned} T^{\{world\}}_i &= T_{align} * T^{\{local\}}_{i,j+1} && \text{(for each frame } i \text{ in chunk } j+1) \\ p_{world} &= R_{align} * p_{local} + t_{align} && \text{(for each point in chunk } j+1) \end{aligned}$$

10.4 Scale-Aware Stitching (Umeyama)

When the monocular scale differs significantly between chunks, rigid welding introduces scale drift. We use the Umeyama similarity transform to correct for this. We extract the camera centres from all o overlap frames in both chunks, giving two sets of corresponding 3D points. The Umeyama solver finds the scale, rotation, and translation that best aligns them.

The complete derivation is given in the Mathematics Breakdown document. The result is a transform $T_{sim} = (s, R, t)$ that maps local points to world points:

$$p_{world} = s * R * p_{local} + t$$

where $s > 0$ is the scale correction, R in $SO(3)$ is the rotation, and t in R^3 is the translation.

10.5 The SVD Crash Bug

The Umeyama solver requires an SVD of the 3×3 cross-covariance matrix. This SVD can fail to converge when the o overlap camera centres are nearly collinear — which happens when the car is driving approximately straight and barely changing direction between the overlap frames. We initially did not handle this and the script crashed on any such chunk. The fix was to wrap the SVD in a try/except block and fall back to the rigid weld when SVD fails.

11. MonST3R — The Dynamic Scene Extension

11.1 The Problem MonST3R Solves

DUST3R assumes the scene is static. In practice, other Formula 1 cars are moving through the scene at up to 300 km/h. DUST3R will attempt to reconstruct these moving cars as if they are static scene elements, placing them in the point cloud at their average position over the chunk duration. This produces smeared, ghostly artefacts in the reconstruction at the positions other cars occupied.

MonST3R addresses this by additionally predicting which pixels belong to moving objects (other cars, marshals, spectators with flags) and down-weighting those pixels in the global alignment energy.

11.2 The Optical Flow Input

MonST3R conditions on a dense optical flow field $F_{a \rightarrow b}$ computed by the RAFT (Recurrent All-Pairs Field Transforms) network. RAFT takes frames I_a and I_b and computes, for every pixel (u,v) in I_a , the displacement to where that pixel's scene point appears in I_b :

$$F_{a \rightarrow b}(u, v) = (\delta_u, \delta_v) \text{ such that } I_b(u+\delta_u, v+\delta_v) \text{ matches } I_a(u, v)$$

Pixels that belong to static scene elements should have flow consistent with the camera's rigid motion. Pixels belonging to moving objects will have flow that deviates from the rigid motion field. MonST3R uses this signal to produce the motion mask M .

11.3 Running MonST3R

MonST3R is run via its `demo.py` script, which handles the full pipeline internally including optical flow computation, sliding window inference, and per-chunk bundle adjustment. The key parameters we use are:

- **--num_frames 90** — Process 90 frames at a time (fits in 16 GB T4 VRAM)
- **--fps 5** — Sample at 5 fps from the input video for initial processing
- **--weights** — Path to the MonST3R checkpoint (approximately 2.3 GB)

MonST3R outputs camera trajectories as text files (`pred_traj.txt`) containing one pose per frame, and depth maps and point clouds for each frame.

12. What Went Wrong — Bugs and Fixes

This section documents every significant bug encountered during the project, its root cause, and how it was resolved. This is the most practically useful section of the document for anyone attempting to reproduce or extend the work.

Bug 1: The Blob — Missing Global Alignment

Symptom: The fused point cloud appeared as a diagonal smear or blob with no recognisable geometry whatsoever.

Root cause: The code extracted 3D points directly from the raw inference output dictionary (scene['pred1'] etc.) without running the PointCloudOptimiser. Every pointmap was in its own arbitrary local coordinate frame, so all pointmaps piled on top of each other at the origin.

Fix: Every chunk must be routed through `global_aligner()` and `compute_global_alignment()` before any points are extracted. The alignment step is mandatory, not optional.

WARNING: Never use raw inference() output as a point cloud. Always run global alignment first.

Bug 2: Empty Chunks — Masking Instead of Cropping

Symptom: 30-35% of chunks produced exactly zero output points. The reconstruction had large gaps in the trajectory corresponding to straight sections and the tunnel.

Root cause: We were masking (zeroing) the lower half of frames instead of cropping them. The network allocated compute to zero-valued pixels, produced near-zero confidence for them, and after confidence thresholding these pixels produced no points. On featureless track sections the remaining upper half pixels also had insufficient texture.

Fix: Physically crop the frame to its upper half. Every pixel the network receives must contain real scene information.

Bug 3: SVD Non-Convergence — Collinear Camera Centres

Symptom: `RuntimeError: SVD did not converge`. Crash occurred when stitching chunks that covered long straight sections.

Root cause: The Umeyama similarity transform requires an SVD of the 3x3 cross-covariance matrix of the overlap camera centres. When the car drives straight for the full overlap region, all 5 camera centres are nearly collinear. The cross-covariance matrix is then nearly rank-1 and the SVD fails to converge numerically.

Fix: Wrap the SVD in a try/except block. If SVD fails, fall back to the rigid weld (identity scale, use `T_align` directly).

Bug 4: PyTorch Weights-Only Load Error

Symptom: `UnpicklingError: Weights only load failed on torch.load()` of the DUST3R checkpoint.

Root cause: PyTorch 2.6 changed the default value of the `weights_only` argument in `torch.load()` from `False` to `True`. The DUST3R checkpoint contains `argparse.Namespace` objects which are not allowed by the restricted unpickler when `weights_only=True`.

Fix: `torch.serialization.add_safe_globals([argparse.Namespace])` before loading any checkpoint. This explicitly allows `argparse.Namespace` to be unpickled while keeping other unsafe classes blocked.

Bug 5: OpenCV PnP Assertion Error

Symptom: `cv2.error: Assertion failed (++num_null_vectors_ <= 6) in function computeOmega`.

Root cause: DUS3R's internal pose initialisation uses OpenCV's SQPnP algorithm to initialise camera poses. This algorithm asserts that the input 3D point set is not degenerate (all points must not lie in a plane). On chunks with very little depth variation (looking down a very flat straight section), this assertion triggers.

Fix: Wrap the entire `reconstruct_chunk()` function in a `try/except` block. Chunks that trigger this error are skipped. They typically correspond to sections with insufficient 3D structure anyway.

Bug 6: Module Import Failure in Colab

Symptom: `ModuleNotFoundError: No module named 'dust3r.inference'`.

Root cause: When `pip install -e .` was run inside the `/content/dust3r` directory, the current working directory became the package root. Python then resolved `import dust3r` to the `/content/dust3r` directory itself rather than the `/content/dust3r/dust3r/` package subdirectory.

Fix: `os.chdir('/content')` and `sys.path.insert(0, '/content/dust3r')` before importing. The change of working directory prevents the directory-as-package confusion.

Bug 7: Recursion Error in torch.load Patching

Symptom: `RecursionError: maximum recursion depth exceeded`.

Root cause: We patched `torch.load` multiple times across different cells in the Colab notebook, each time wrapping the already-patched version. The n -th patch called the $(n-1)$ -th patch which called the $(n-2)$ -th patch and so on, producing infinite recursion.

Fix: Store the true original before the first patch: `torch._ORIGINAL_LOAD = torch.load`. Always patch from the stored original, never from the current `torch.load`.

13. Results — What We Produced

13.1 Per-Chunk Statistics

Chunk type	Example turns	Alignment loss	Points per chunk	Quality assessment
Slow corner, rich texture	Turns 3-5, 8	0.004 - 0.007	100,000 - 190,000	Excellent. Matches ground truth.
Medium-speed corner	Turns 1, 10, 18	0.005 - 0.010	30,000 - 100,000	Good. Minor artefacts at apex.
Fast straight	Pit straight	0.001 - 0.002	0 - 5,000	Poor. Insufficient texture and parallax.
Tunnel section	Turns 10-12	0.001	0	Failed. No texture in darkness.
Heavy traffic (lap 1)	Multiple	0.003 - 0.008	5,000 - 50,000	Moderate. Cars corrupt geometry.

13.2 Evolution of Output Quality

The reconstruction output went through four distinct phases as we fixed bugs and improved the pipeline:

- **Phase 1 (initial code)** — Diagonal blob. No recognisable geometry. Caused by missing global alignment.
- **Phase 2 (alignment added, masking retained)** — Mostly empty. 35% of chunks produced zero points. Some chunks showed partial road geometry.
- **Phase 3 (cropping applied)** — Major improvement. Chunks 0-2 produce 154k, 132k, 104k points. Track geometry visible in individual chunk views.
- **Phase 4 (full pipeline, error handling)** — Stitched global point cloud with continuous camera path. Some sections still sparse (straights, tunnel). Per-turn geometry matches known Monaco circuit layout.

13.3 What Good Output Looks Like

A correctly reconstructed chunk covering a slow corner shows, when viewed from above in Open3D:

- Two parallel dense ridges of points corresponding to the left and right barriers.
- A central lower-density band corresponding to the track surface.
- Scattered points above track level corresponding to grandstands, trees, and safety fencing.
- A smooth curved blue ribbon of camera trajectory points threading through the track centre.

The curvature of the trajectory matches the known corner geometry of Monaco. For turns 3, 4, and 5 (the Casino square section), our per-chunk reconstruction visually matches the ground truth aerial photography of these corners.

13.4 Final Output Files

- **vroom_global.ply** — The globally fused 3D point cloud of the circuit. Points coloured red-to-green by confidence. File size approximately 50-200 MB depending on point count.

-
- **vroom_camera_path.ply** — The recovered racing line. One blue point per processed frame. Smooth curved path corresponding to the optimal trajectory through each corner.
 - **chunk_XXX_aligned.ply** — Individual chunk reconstructions. 88 files, one per chunk. Useful for diagnosing which sections of the lap reconstruct well and which do not.

14. Lessons Learned

14.1 Preprocessing Is Load-Bearing

The most important lesson from this project is that preprocessing is not cosmetic. The single biggest quality improvement came not from changing the reconstruction model or the alignment algorithm but from changing how we removed the car body — switching from masking to cropping. This one change increased per-chunk point counts from 0 on 35% of chunks to 100,000+ on the same chunks. The network can only work with what it receives. Sending it informative pixels is more valuable than tuning the alignment iterations.

14.2 Global Alignment Is Mandatory

The second most important lesson: never use raw inference output as a point cloud. The global alignment step is the mechanism that converts a pile of disconnected local coordinate systems into a coherent 3D reconstruction. It cannot be skipped or shortcut. This was the primary bug in our codebase and cost significant debugging time before the root cause was identified.

14.3 Test on Fewer Chunks First

We wasted significant compute time running the full 88-chunk pipeline while both the masking bug and the alignment bug were active. The blob output looked different from what we expected but we assumed it was a stitching issue and kept running more chunks. We should have run a single chunk in isolation first, inspected the output, confirmed it showed recognisable geometry, and only then moved on to stitching. Testing end-to-end on one unit before scaling up would have saved hours.

14.4 Error Handling Is Essential for Long Pipelines

With 88 chunks, it is inevitable that some will fail due to degenerate geometry, numerical issues, or unusual scene content. A pipeline that crashes on the first bad chunk will never complete. Every chunk-level operation must be wrapped in error handling that skips the offending chunk and continues. The final pipeline can tolerate up to 30-40% of chunks failing without producing an unusable global reconstruction.

14.5 The Right Tool for the Right Environment

We initially ran the pipeline on a Windows machine with a GTX 1650 (4 GB VRAM). This was fundamentally insufficient for DUST3R's ViT-Large model, which requires 6-10 GB for a 12-frame chunk. The paging file workaround we attempted caused system crashes. The correct solution was to use a cloud GPU (Colab T4, 16 GB) from the beginning. Choosing the right compute environment before starting the pipeline work would have saved significant setup time.

15. Future Work

15.1 Loop Closure

The highest-priority future direction is loop closure. Sequential chunk stitching accumulates error; after 88 chunks the trajectory end does not coincide with the trajectory start even though they represent the same physical location. Pose-graph optimisation with a loop closure constraint would jointly optimise all chunk transforms to minimise this accumulated drift while enforcing that the lap ends where it began.

15.2 MonST3R Full Pipeline

We have set up the MonST3R infrastructure but have not yet run the complete end-to-end MonST3R pipeline on the full video. MonST3R's optical flow conditioning and motion masks should substantially improve reconstruction quality in sections where other cars are visible. This is particularly important for lap 1 when the field is tightly packed.

15.3 Multi-Car Fusion

We have 20 onboard videos — one per driver. Combining the point clouds from multiple drivers would dramatically improve reconstruction quality in two ways. First, each driver sees sections of track that others do not see as clearly (the car behind another car has a different view of that section than the car ahead). Second, multiple independent viewpoints can resolve the car-body occlusion problem without masking: one car's camera can see the track that another car's chassis is blocking.

15.4 Camera Calibration

DUSt3R estimates focal length per-frame during global alignment. This is less accurate than having known intrinsics from a calibration procedure. If the F1 broadcast cameras were calibrated (their focal length and distortion coefficients measured), this information could be provided to DUSt3R as a fixed constraint rather than a free parameter, improving reconstruction accuracy.

15.5 Quantitative Evaluation

We currently evaluate reconstruction quality visually and via alignment loss values. A proper quantitative evaluation would compare our reconstructed track geometry to the official FIA circuit map (which defines the precise track boundaries, corner radii, and elevation changes) and report accuracy in metres. This would give a rigorous quality metric and allow systematic comparison of different preprocessing and model choices.
